Runtime and Memory Scaling of Naive Dense Matrix Multiplication in Python, Java and C

Big Data – Individual Assignment 01

Daniel Rodríguez Alonso
Grado en Ciencia e Ingeniería de Datos
Universidad de Las Palmas de Gran Canaria (ULPGC)
Big Data, Academic Year 2026–2027

***Abstract*—**

***Index Terms*—matrix multiplication, benchmarking, performance evaluation, programming languages, JIT compilation, memory hierarchy, peak resident set size**

I. Introduction and Objective

Dense matrix multiplication is a building block of many data-intensive workloads (linear models, similarity computations, neural-network layers). Its classical definition needs n^3 multiply–add operations for two $n \times n$ matrices, so the cost of an implementation grows quickly with the data size, and the constant factor in front of n^3 is decided by the language runtime, the data layout and the memory hierarchy. This report is the first of four related studies; it establishes a validated sequential baseline that the later assignments (optimisation, sparse matrices, parallel and distributed execution) reuse.

Research question (RQ). *On one Intel Core i7-9750H laptop, how do the runtime and the memory use of the same naive triple-loop float64 matrix multiplication grow with the matrix size n when it is implemented in C, Java and Python, and how large are the differences between the three implementations at equal n?* The factor varied is n (side length of square $n \times n$ matrices) for each of three implementations; the outcomes are the kernel runtime, the time per multiply–add, the peak memory of the process and the largest feasible n under fixed budgets.

Hypotheses (predictions stated before the measurements). *H1:* the runtime of every implementation grows approximately as n^3 (fitted exponent $b \approx 3$), with possible deviations at large n when the working set leaves the caches. *H2:* at equal n the order is C $<$ Java $<$ Python; after just-in-time (JIT) warm-up Java is within a small factor of C, while Python is one to two orders of magnitude slower. *H3:* Python needs several times more memory per matrix element than C and Java, because every element is a boxed object.

The study contributes (i) three implementations with identical loop order that produce bit-identical results, (ii) a protocol with shared input files, a common timing boundary and recorded failures, and (iii) runtime, growth-exponent and memory measurements with their variability.

II. Background and Related Work

Classical algorithm. For $C = AB$, $c_{ij} = \sum_{k=0}^{n-1} a_{ik} b_{kj}$, which costs n^3 multiplications and n^3 additions ($2n^3$ floating-point operations); the three nested loops can be written in six orders that perform the same arithmetic but access memory differently [1]. In the ijk order with row-major storage used here, the innermost loop reads a row of A with unit stride and a column of B with stride n, i.e. every access to B touches a different cache line. Once B ($8n^2$ bytes) no longer fits in a cache level, the column walk causes cache and translation-lookaside-buffer (TLB) misses; Drepper uses exactly this matrix multiplication to illustrate the effect of access patterns on the memory hierarchy [2], and Lam et al. show that cache interference in matrix kernels depends strongly on the access stride and the matrix size [3]. High-performance libraries avoid these costs with blocking and packing [4]; such optimisations are deliberately excluded here and are the subject of Assignment 2.

Language runtimes. C is compiled ahead of time to machine code. Java bytecode is first interpreted and then compiled at run time by the HotSpot JIT compiler, so early iterations are slower and timings vary between virtual-machine invocations; Georges et al. recommend steady-state measurements and several VM invocations for this reason [5]. Since JDK 17 all Java floating-point operations are strict [6]. CPython 3.12 executes bytecode in an interpreter that specialises instructions adaptively at run time but does not generate machine code [7]; each Python `float` is a separate heap object, and a `list` stores references to such objects. `sys.getsizeof` reports only the memory directly attributed to an object, not to the objects it refers to [8].

Numerical accuracy. A dot product of length n computed in floating point satisfies $|\hat{c}_{ij} - c_{ij}| \leq \gamma_n (|A||B|)_{ij}$ with $\gamma_n = nu/(1-nu)$ and unit round-off $u = 2^{-53}$ for float64 [9]. Different summation orders therefore give slightly different results, while the same order and the same IEEE 754 operations give identical results.

Benchmarking methodology. Hoefler and Belli recommend reporting medians and non-parametric variability measures, the number of repetitions and the full experimental setup [10]; Mytkowicz et al. show that seemingly innocuous aspects of the setup can bias conclusions [11], which motivates scoping

the conclusions to the tested environment.

III. Methodology

A. Implementations

The three programs (`src/c`, `src/java`, `src/python`) implement the same kernel:

```
for i in 0..n-1:
  for j in 0..n-1:
    s = 0.0
    for k in 0..n-1: s += A[i][k] * B[k][j]
    C[i][j] = s
```

with float64 values (`double` in C and Java, `float` in Python), the same ijk order and a scalar accumulator, so the floating-point evaluation order is identical. Each language uses its idiomatic storage, which is the main unavoidable difference: *C* stores each matrix in one contiguous row-major block (`malloc`/`calloc`, element (i, j) at `M[i*n+j]`); *Java* uses `double[][]`, an array of separately allocated row arrays, so `b[k][j]` first loads the reference to row k and then the element, with bounds checks managed by the JIT; *Python* uses a list of row lists of `float` objects, so every access dereferences a pointer to a boxed value and every multiply–add is executed by the interpreter and allocates a new `float`. Java and Python hoist the row references `A[i]` and `C[i]` out of the inner loops; C computes the flat index. No library routine is used in any kernel; NumPy [12] is used only to generate the inputs and as the reference in the correctness tests.

Fixed settings. C: gcc 13.3.0 with `-O2 -std=c11 -Wall -Wextra`, default x86-64 target (SSE2, no `-march`), no `-ffast-math`, single thread; the generated assembly of the kernel contains only scalar `mulsd`/`addsd` instructions (no vectorisation and no fused multiply–add, FMA). Java: OpenJDK 21.0.12.1, `javac -release 21`, `java -Xms1g -Xmx4g -XX:+UseG1GC`, default tiered JIT. Python: CPython 3.12.3, standard library only. The kernel runs on one thread in every language (the JVM keeps its background JIT and garbage-collection threads); `OMP_NUM_THREADS`, `OPENBLAS_NUM_THREADS` and `MKL_NUM_THREADS` are set to 1 for every run.

B. Inputs

All programs read the same files, generated once by `scripts/gen_inputs.py`: NumPy `default_rng(42)` (PCG64), A drawn first and then B, independent uniform values in $[-1, 1)$, stored as raw little-endian float64 in row-major order with a JSON sidecar and SHA-256 checksums. Since $|c_{ij}| \leq n$, no overflow can occur. Loading the files, building the language-specific structure and allocating C happen before the timed region.

C. Experimental environment

Table I summarises the machine. The laptop was shared with three other benchmark sessions of related assignments; all timed runs were serialised by a machine-wide file lock (Section III-E).

TABLE I
Execution environment (`results/environment.md`, captured by `capture_env.sh`).

Item	Value
CPU	Intel Core i7-9750H, 1 socket, 6 cores / 12 threads, 2.6 GHz base, 4.5 GHz max, AVX2/FMA
Caches	L1d 32 KiB and L2 256 KiB per core; L3 12 MiB shared
Frequency	governor `powersave`, turbo enabled, AC power connected
Memory	15 GiB (16.6 GB) RAM
OS	Ubuntu 24.04.5 LTS, Linux 7.0.0-38-generic, x86-64
C	gcc 13.3.0, `-O2 -std=c11 -Wall -Wextra`
Java	OpenJDK 21.0.12.1 (HotSpot 64-bit Server VM), `-Xms1g -Xmx4g -XX:+UseG1GC`
Python	CPython 3.12.3 (venv)
Tools	NumPy 2.5.4 (inputs, reference), GNU `time` (RSS)
Threads	kernel single-threaded; `OMP/OPENBLAS/MKL_NUM_THREADS=1`

D. Correctness validation

Before any benchmark, `tests/test_correctness.py` runs the three benchmarked programs as black boxes and checks (**??**): known products with exact answers (1×1, 2×2 and a non-symmetric 3×3 with small integers), $AI = A$, $IB = B$ and $A \cdot 0 = 0$ for $n = 7$ (exact), and random uniform inputs for $n \in \{7, 64, 100\}$ against NumPy with the element-wise tolerance $|c_{ij} - c_{ij}^{\text{ref}}| \leq 2nu(|A||B|)_{ij}$. The factor 2 accounts for both results carrying an error of at most $\gamma_n(|A||B|)_{ij} \approx nu(|A||B|)_{ij}$, since NumPy sums in a different order. Results of the three own implementations must be bit-identical, because they perform the same IEEE 754 operations in the same order. During the benchmark the checksum $\sum_{ij} c_{ij}$ printed after every repetition is compared across languages at every n.

E. Measurement protocol and budgets

Timer and boundary. Each program measures the kernel only (boundary *kernel*) with a monotonic clock: `clock_gettime(CLOCK_MONOTONIC)` in C, `System.nanoTime()` in Java and `time.perf_counter_ns()` in Python. Input loading, allocation and zeroing of C and the checksum are excluded. Printing the checksum consumes the result, which prevents dead-code elimination. For short kernels a sample times a batch of $\lceil 0.2\,\text{s}/t_{\text{w}} \rceil$ consecutive calls (t_{w}: last warm-up time) and reports the time per multiplication; the batch size is stored with every row.

Repetitions and warm-up. A configuration (language, n) consists of P separate process runs; each process performs W in-process warm-up repetitions (recorded, excluded from statistics) and one measured repetition, giving $k = P$ samples that include between-process variability [5]. A pilot (`results/raw/pilot_size_sweep.csv`) showed a strong JIT effect in Java and no warm-up effect in C at $n = 1024$ or in Python at $n = 512$. Within the 90-minute budget this led to: C $W = 2$ ($n \leq 512$), 1 ($n = 768$), 0 ($n \geq 1024$); Java $W = 5$ ($n \leq 512$), 2 ($n \leq 1024$), 1

($n \geq 1536$); Python $W = 1$ ($n \leq 512$), 0 ($n \geq 768$); $P = 5$ everywhere except Python at $n = 1536$ ($P = 3$, about six minutes per run) and $n = 2048$ (one attempt to document the time limit). The order of the languages was shuffled at every n (fixed seed).

Memory. The common metric is the *peak resident set size* (RSS) of the whole process, measured by GNU `time` (`%M`) [13] for every process run; it includes the interpreter or virtual machine. To separate this runtime baseline, the same programs were run at $n = 1$ ($k = 5$). Each program also reports one language-internal metric, which is not comparable across languages: C `matrix_storage_bytes` (the exact $3 \cdot 8n^2$ bytes allocated), Java `jvm_heap_peak_used_bytes` (sum of the peak usage of the heap `MemoryPoolMXBeans` [14]) and Python `pyobj_deep_size_bytes` (`sys.getsizeof` of the outer lists, row lists and every element, measured outside the timed region).

Sizes, budgets and stopping rule. Declared sequence $n \in \{64, 128, 256, 384, 512, 768, 1024, 1536, 2048\}$, mixing powers of two with $384 = 3 \cdot 2^7$, 768 and 1536. Budgets fixed before the large runs: 600 s per process run, 8 GB of memory, at most 90 minutes of benchmark time for the whole study. Dense storage needs $3 \cdot 8n^2$ bytes (96 MiB at $n = 2048$), so memory is not the limiting resource. A run that exceeds a limit is recorded with its status and an empty time, and larger sizes of that language are not attempted.

Execution and statistics. Every configuration ran through `run_measured.py`, which holds a machine-wide lock while measuring, records the process wall time, peak RSS, exit status and the 1-minute load average, and appends one raw CSV row per repetition. The main sweep ran from 2026-10-27 22:48 to 22:54 (300 raw rows, 215 of them warm-ups). For each configuration the report gives the median and the interquartile range (IQR $= Q_3 - Q_1$) of the k measured samples. No measurement was excluded. The growth exponent b in $T \approx c\, n^b$ is fitted by least squares on $\log T$ vs. $\log n$ of the medians, with a 95 % confidence interval from the Student t distribution; runtime ratios at equal n use the ratio of medians with a 95 % bootstrap interval (5000 resamples).

IV. Results

V. Discussion

VI. Conclusions

Reproducibility Materials

Repository: https://github.com/danirodriguezz/bigdata-assignment01-matmul-languages (submitted version: tag `v1.0-submission`); the same content is in `Assignment01_RodriguezAlonso_Materials.zip`. It contains the source code (`src/`), the correctness tests (`tests/`), the input generator and experiment driver (`scripts/`), the configuration (`configs/experiment.json`), all raw measurements (`results/raw/*.csv`, schema in `results/README.md`), the environment snapshot, the analysis scripts that produce every table, figure and number of this report (`analysis/make_all.py`) and this report's source. `make setup test inputs bench analysis report` reproduces the study; `make analysis report` regenerates all tables and figures from the committed raw data.

Use of Generative AI

Claude Code (Anthropic), model Claude Opus 5.5, was used as an assistant to plan the experiment, to draft and debug the C, Java and Python programs, the correctness tests, the benchmark driver and the analysis scripts, to draft the text of this report and the README, and for language revision. All measurements were produced by runs on the author's laptop and are stored in the repository; no result was generated or estimated by the tool. Every reference was checked against its DOI or publisher record or its official URL. The author reviewed the code, the data and the text and is responsible for the content.

References

[1] G. H. Golub and C. F. V. Loan, *Matrix Computations*, 4th ed. Baltimore, MD: Johns Hopkins University Press, 2013.

[2] U. Drepper, "What every programmer should know about memory," Red Hat, Inc., technical report, 2007, https://www.akkadia.org/drepper/cpumemory.pdf (accessed 2026-10-27).

[3] M. D. Lam, E. E. Rothberg, and M. E. Wolf, "The cache performance and optimizations of blocked algorithms," in *Proc. 4th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS-IV)*. ACM, 1991, pp. 63–74.

[4] K. Goto and R. A. van de Geijn, "Anatomy of high-performance matrix multiplication," *ACM Transactions on Mathematical Software*, vol. 34, no. 3, pp. 1–25, 2008.

[5] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*. ACM, 2007, pp. 57–76.

[6] OpenJDK, "JEP 306: Restore always-strict floating-point semantics," OpenJDK JDK Enhancement Proposal, delivered in JDK 17, https://openjdk.org/jeps/306 (accessed 2026-10-27).

[7] M. Shannon, "PEP 659 – specializing adaptive interpreter," Python Enhancement Proposals, 2021, https://peps.python.org/pep-0659/ (accessed 2026-10-27).

[8] Python Software Foundation, "sys — system-specific parameters and functions (`sys.getsizeof`), Python 3.12 documentation," https://docs.python.org/3.12/library/sys.html (accessed 2026-10-27).

[9] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA: Society for Industrial and Applied Mathematics (SIAM), 2002.

[10] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC '15)*. ACM, 2015, pp. 1–12.

[11] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!" in *Proc. 14th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS XIV)*. ACM, 2009, pp. 265–276.

[12] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.

[13] Linux man-pages project, "time(1) — run programs and summarize system resource usage," https://man7.org/linux/man-pages/man1/time.1.html (accessed 2026-10-27).

[14] Oracle, "Interface MemoryPoolMXBean, Java SE 21 & JDK 21 API specification," https://docs.oracle.com/en/java/javase/21/docs/api/java.management/java/lang/management/MemoryPoolMXBean.html (accessed 2026-10-27).